

Companion di studio, Modulo 4 (materiale extra per l'orale)

Materiale di studio personale, separato dal report consegnato (che non viene toccato). Copre: le risposte ai dubbi del todo, il walkthrough del report, i sei esperimenti aggiuntivi (01..06_*/), i deep dive su MPI nel codice, i punti di onestà e il cheat-sheet finale.

Tutti i numeri sono misurati: o dai CSV del report (results/cluster/), o dagli esperimenti extra (0N_*/results/), girati sugli stessi nodi Ivy Bridge (E5-2640 v2, 32 CPU logiche, 2 NUMA) con job SLURM esclusivi fino a 8 nodi (job 696527-696532). Parametri del report: NR=50M, NS=100M, P=256, seed 42, max_key=25M; baseline sequenziale 4.48 s (uniforme) e 2.30 s (skewed), ricompilata con `-march=ivybridge`.

1. Risposte rapide ai dubbi del todo

Perché la baseline sequenziale è ricompilata con `-march=ivybridge`? La compilazione avviene sul login node, che ha AVX2; i compute node sono Ivy Bridge e non lo hanno. Con `-march=native` sul login il binario userebbe istruzioni illegali sui compute node (o, se compilato lì, non sarebbe riproducibile dal login). `-march=ivybridge` fissa l'ISA del target. Con `-march=native` su una macchina AVX2 il compilatore potrebbe vettorizzare in modo più aggressivo histogram e hash, ma il kernel è memory-bound: il guadagno atteso sarebbe comunque piccolo.

Perché la baseline skewed (2.30 s) è più veloce dell'uniforme (4.48 s)? Località, non meno lavoro: con il 90% dei record su poche partizioni lo scatter scrive in pochi stream quasi contigui invece che in 256 sparsi (nel breakdown sequenziale gli scatter calano di circa 3.4x), e le tabelle hot hanno pochissime chiavi distinte e stanno in cache. L'output (join_count) è anzi più grande, non più piccolo.

Le partizioni hot sono 4 o 8? Fino a 8. R e S sono generate con seed diversi (seed e seed ^ S_SEED_OFFSET), quindi le 4 hot key di R e le 4 di S sono diverse e cadono su partizioni diverse (salvo coincidenze). Le 4 hot di S pesano $0.225 \times 100M = 22.5M$ record l'una, le 4 di R $0.225 \times 50M = 11.3M$. Il rank più carico a 128 rank riceve infatti 22.6M record (misurato, esp. 4): una hot di S più il rumore uniforme.

Da dove viene il "hard floor" sotto skew, in numeri? Esp. 4: a 128 rank il rank più carico riceve 22.6M record contro una media di 1.2M (19x). La partizione hot più pesante è indivisibile con questo schema di ownership: nes-sun mapping partizione-rank può scendere sotto quel massimo. Per questo il remapping non basta e servirebbe replicare il build side (sez. 4.5 del report).

Il remapping consapevole dei pesi conviene? Dipende dal regime (esp. 4): a 8 rank sì (le hot collidono sotto pid mod R: 58.1M sul rank peggiore; il greedy le separa e dimezza il totale, 0.88 contro 1.70 s, con un costo di piano di 2-3 ms); a 128 rank no (il pavimento è già raggiunto da mod, il greedy non guadagna nulla sul join e peggiora la collettiva del 2.3x perché i send count diventano irregolari). La scelta del report (mapping semplice, niente remap) è giusta nel regime in cui il report opera.

Perché MPI_Barrier prima di ogni collettiva? Non falsa la misura? Il barrier allinea i rank così il timer della collettiva misura lo scambio e non l'attesa del rank in ritardo dalla fase precedente. Esp. 4: senza barrier comm_sizes assorbe 3-6 ms di skew di arrivo e il totale cambia meno del 5%. La riduzione MAX per fase mostra il rank più lento, cioè il critical path di quella fase. Metodologia difendibile e con effetto marginale sul totale.

FUNNELED vs MULTIPLE: quanto costa davvero MULTIPLE? Zero misurabile qui (esp. 6: 0.49 contro 0.50 s su uniforme, dentro il rumore). Il locking interno di MULTIPLE pesa solo se più thread chiamano MPI concorrentemente; in questo codice ogni chiamata MPI sta fuori dalle regioni OpenMP. La motivazione onesta di FUNNELED: chiedere il livello minimo documenta il contratto (solo il main thread parla con MPI) e non lascia costi latenti, non che MULTIPLE renda questa pipeline più lenta.

Perché P=256? Vincoli: P multiplo di ogni rank count usato (fino a 256 rank in pure MPI, quindi P almeno 256). Esp. 5: per l'ibrido P conta poco (il join migliora con P per via delle tabelle in cache, lo scambio no); per il pure MPI su uniforme P=1024 sarebbe meglio (0.71 contro 1.12 s a P=128), su skewed peggio oltre 256. P=256 è il compromesso che copre tutte le configurazioni con un solo valore.

comm_sizes: perché 10-13 ms da 2 nodi in su se scambia pochi byte? L'Alltoall dei contatori muove R interi per rank, ma è comunque una collettiva a R partecipanti: paga la sincronizzazione. In più qualunque residuo di

disallineamento dopo il barrier si scarica sul primo che si blocca. Esp. 4 (barrier off) lo conferma: la fase cresce a 4-7 ms extra proprio quando il barrier non c'è. Resta comunque due ordini di grandezza sotto il payload.

Lo skip-ahead di splitmix64. Lo stato di splitmix64 avanza di una costante fissa (la sezione aurea a 64 bit) a ogni estrazione: lo stato dopo k estrazioni è $\text{seed} + \text{GOLDEN} * k$, calcolabile in $O(1)$. Ogni rank quindi genera solo la propria fetta partendo da $\text{seed} + \text{GOLDEN} * \text{offset}$, e la concatenazione delle fette è byte per byte la sequenza del generatore sequenziale. Per lo skewed non si può: la selezione iniziale delle hot key fa rejection sampling (numero di estrazioni variabile), quindi l'offset per-record non è noto e ogni rank rigenera l'intera relazione e ne taglia la fetta, fuori dalla regione misurata.

Perché il naive verifier è una prova forte? È un doppio ciclo $O(NR \times NS)$ senza hash, senza partizionamento e senza codice condiviso con il kernel: un bug del kernel non può riprodursi identico in un'implementazione indipendente, quindi la coincidenza della tripla (join_count, ck1, ck2) su input piccoli è un test severo. Sui 46 casi di validazione: piccoli con naive, medi e grandi contro hash_join_seq, rank da 1 a 128, con il vincolo $P \geq R$ (ogni rank deve possedere almeno una partizione) che sui piccoli input ($P=16$) limita i rank a 8.

2. Walkthrough del report

Pipeline a 8 fasi. histogram_local e scatter_local preparano il buffer di invio in ordine dest-major (blocchi contigui per destinatario: requisito di Alltoallv); comm_sizes scambia i contatori per dimensionare esattamente i buffer di ricezione; comm_payload è l'Alltoallv dei record; histogram_post e scatter_post rimettono il buffer ricevuto in layout per-partizione così il kernel del Modulo 3 si riusa senza modifiche; join_local fa build+probe sulle partizioni possedute; reduce_final è un Allreduce dei tre aggregati in $O(\log R)$ round.

Strong scaling (fig. 1, tab. 1). Ibrido monotono da 2.8x (1 nodo) a 12.0x (8 nodi). Pure MPI: picco 7.6x a 2 nodi, crollo a 3.0x a 4 nodi (128 rank), recupero a 7.1x a 8. Il crollo è tutto in comm_payload: 0.09 -> 0.29 -> 1.35 -> 0.53 s. La catena di prova del report: (a) 128 rank su 8 nodi mostrano la stessa fase lenta e rumorosa, quindi è il rank count e non i nodi; (b) basic linear forzato è stabile ma lento, quindi la varianza viene dalla selezione dell'algoritmo. L'esp. 1 aggiunge: (c) il fenomeno si riproduce con buffer sintetici senza pipeline, e (d) pairwise forzato è insieme veloce e stabile (0.13 s a 8 nodi contro 0.40 del default): la "cura" esisteva, un parametro MCA.

Weak scaling (fig. 2). Ibrido 0.58 di efficienza a 8 nodi, pure MPI 0.21. Il report lo spiega con il modello di Hockney: per-rank $(R-1) \alpha + \beta V$, con il termine banda costante per costruzione, quindi crescono gli startup. L'esp. 2 corregge il meccanismo: con α misurato (22 us) gli startup valgono 5.6 ms a 256 rank, non 3.1 s; ciò che cresce con i rank per nodo è il volume che attraversa ogni scheda di rete (32 rank/nodo = 32 x 48 MB per NIC). Il modello "beta x volume off-node per nodo" segue la curva; la conclusione (meno partecipanti = meglio) resta, il termine dominante è la banda per NIC.

Breakdown a 4 nodi (fig. 3). Pure MPI uniforme: 88% del tempo in comm_payload, fasi locali ~0.07 s (ogni rank tocca 1/128 dell'input). Ibrido: fasi locali qualche decina di ms l'una (OpenMP), comm_payload al 44% ma su un totale di 0.61 contro 1.30 s. Skewed: pure MPI metà tempo in comunicazione e il resto nelle fasi post (i rank hot rifanno histogram e scatter su 20x i record medi); l'ibrido soffre nel join locale perché la partizione hot è una sola iterazione del loop dynamic e un thread la macina da solo. Nota: nell'ibrido a 4 rank fino a 4 delle hot possono capitare su rank diversi, ma dentro il rank la hot resta indivisibile a livello di schedule.

Skew (fig. 4). L'ibrido rallenta poco; il pure MPI non scende mai sotto il tempo a 1 nodo. Il punto concettuale: in shared memory il dynamic schedule può bilanciare il volume (ogni thread vede tutte le partizioni); dopo l'Alltoallv ogni rank possiede fisicamente le sue partizioni e nessuno può rubare lavoro senza muovere dati. Il farm a stato partizionato con hashing bilancia il numero di partizioni, non il loro volume. Esp. 4 quantifica il pavimento (19x) e mostra che il remapping non lo rimuove.

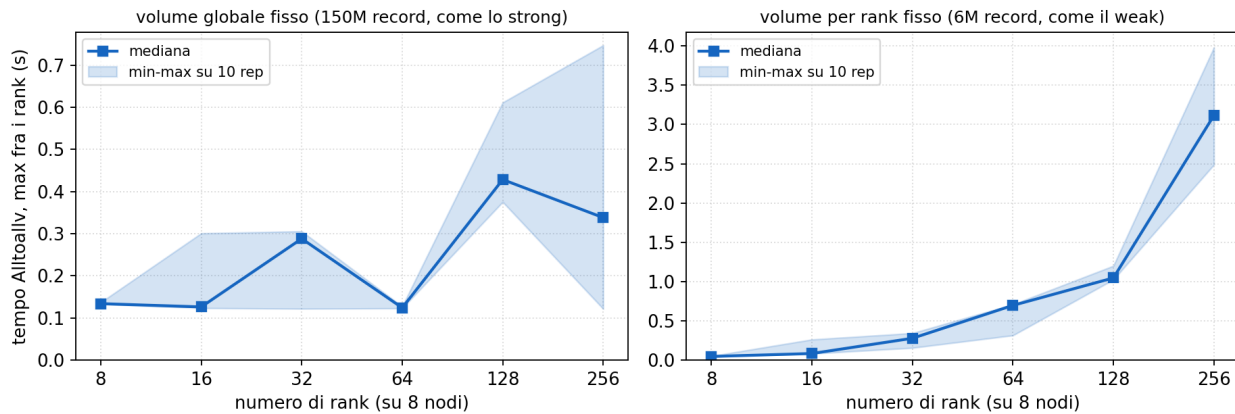
Cross-module (fig. 5, tab. 3-4). Uniforme: M3 su 1 nodo 10.1x; M4 ibrido su 8 nodi 12.0x: otto volte l'hardware per +20%. Il collo condiviso: il join è memory-bound e la collettiva cresce con i rank. Skewed: M3 6.2x contro M4 2.5x, per la ragione del punto precedente. Conclusione del report: la distribuzione si giustifica per capacità (input che non sta su un nodo), non per velocità su questo input da 1.2 GB.

Ibrido (sez. 6). Prima iterazione: solo il join era parallelo dentro il rank e a 1 nodo faceva 6.25 s, peggio del sequenziale (Amdahl: histogram e scatter seriali su 150M record). Parallelizzare tutte le fasi locali porta a 1.61 s. Il calo di rank (256 -> 8) taglia l'Alltoallv da 0.53 a 0.18 s a parità di payload.

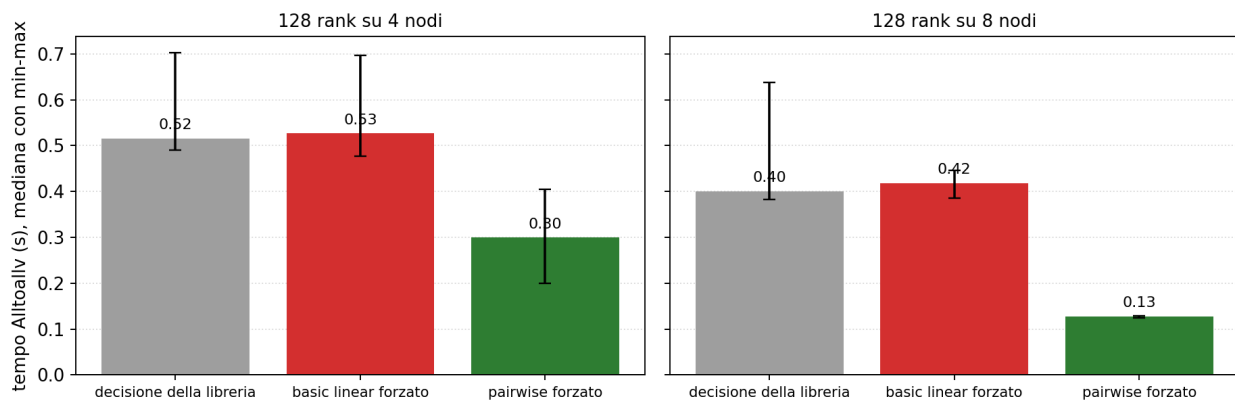
3. I sei esperimenti (spiegazione, meccanismo, conferma)

Esp. 1: anatomia di MPI_Alltoallv (01_alltoallv_anatomy/)

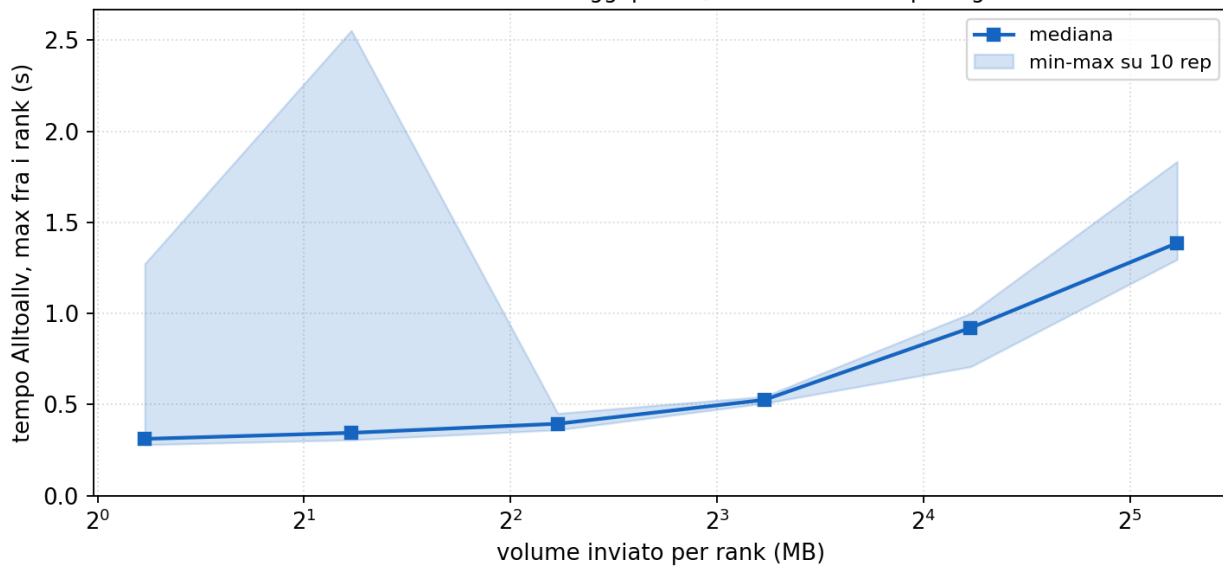
Alltoallv sintetico: il costo segue il rank count, non i nodi



Algoritmo della collettiva forzato via MCA (stesso volume del caso strong)



Sweep del volume a 128 rank su 4 nodi (algoritmo default):
varianza enorme sui messaggi piccoli, costo lineare su quelli grandi

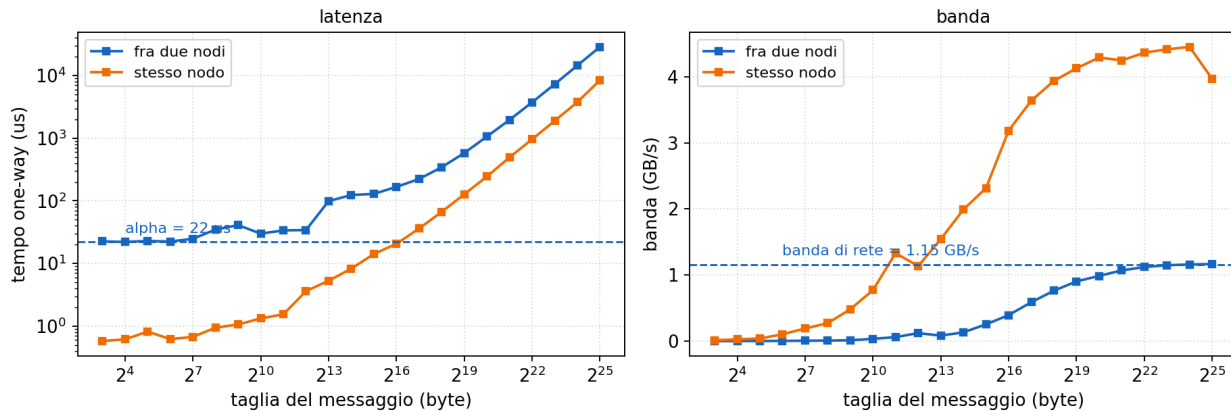


Microbenchmark (`common/alltoallv_bench.cpp`): solo lo scambio, buffer sintetici, warm-up più 10 rep misurate con barrier e riduzione max. Tre risultati:

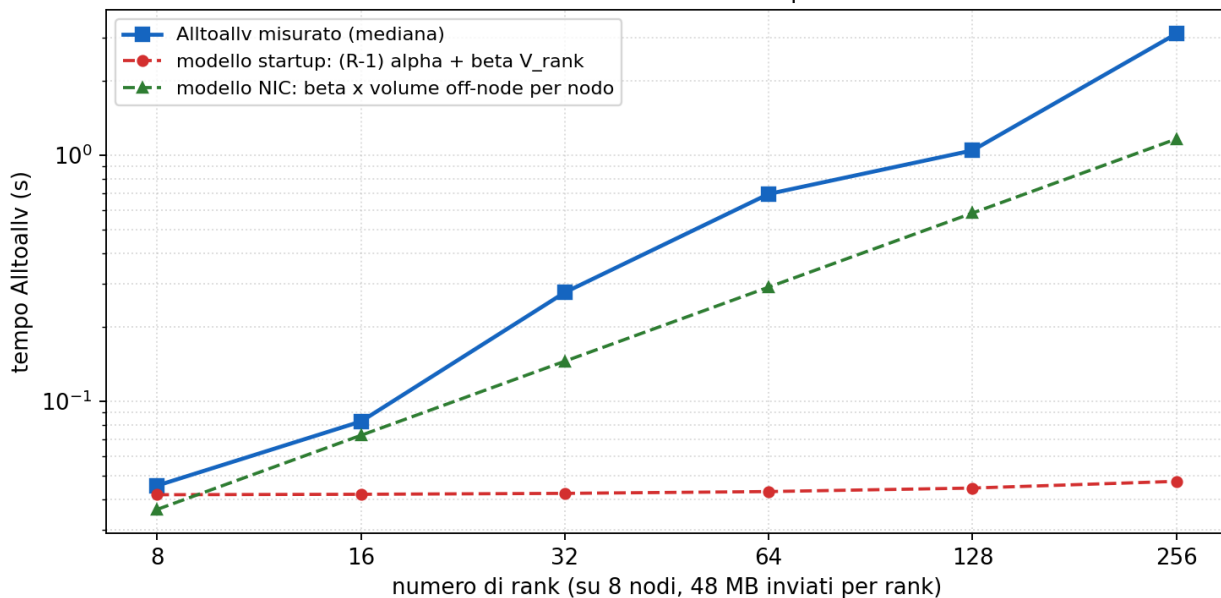
1. **Il dip è del rank count.** A volume globale fisso su 8 nodi: 64 rank 0.12 s, 128 rank 0.43 s con metà dei dati per rank, 256 rank mediana 0.34 s ma spread 0.12-0.75. Riprodotto senza join né generatore.
2. **Pairwise è la cura.** Forzando `coll_tuned_alltoallv_algorithm=2`: 0.30 contro 0.52 s (4 nodi), 0.13 contro 0.40 s (8 nodi, stabile). La decisione automatica in questo regime si comporta come basic linear. Openmpi sceglie l'algoritmo con euristiche su taglia e participant count tarate su altri sistemi: qui sbaglia. Pairwise scambia in R-1 round strutturati (al passo k il rank r manda a r+k e riceve da r-k): niente incast, traffico bilanciato per round.
3. **La varianza vive sui messaggi piccoli.** Volume sweep a 128 rank: sotto ~4 MB per rank il min-max esplode (0.15-2 s); sopra, il costo cresce linearmente col volume.

Esp. 2: Hockney misurato (02_hockney/)

Ping-pong: i coefficienti del modello di Hockney misurati sul cluster



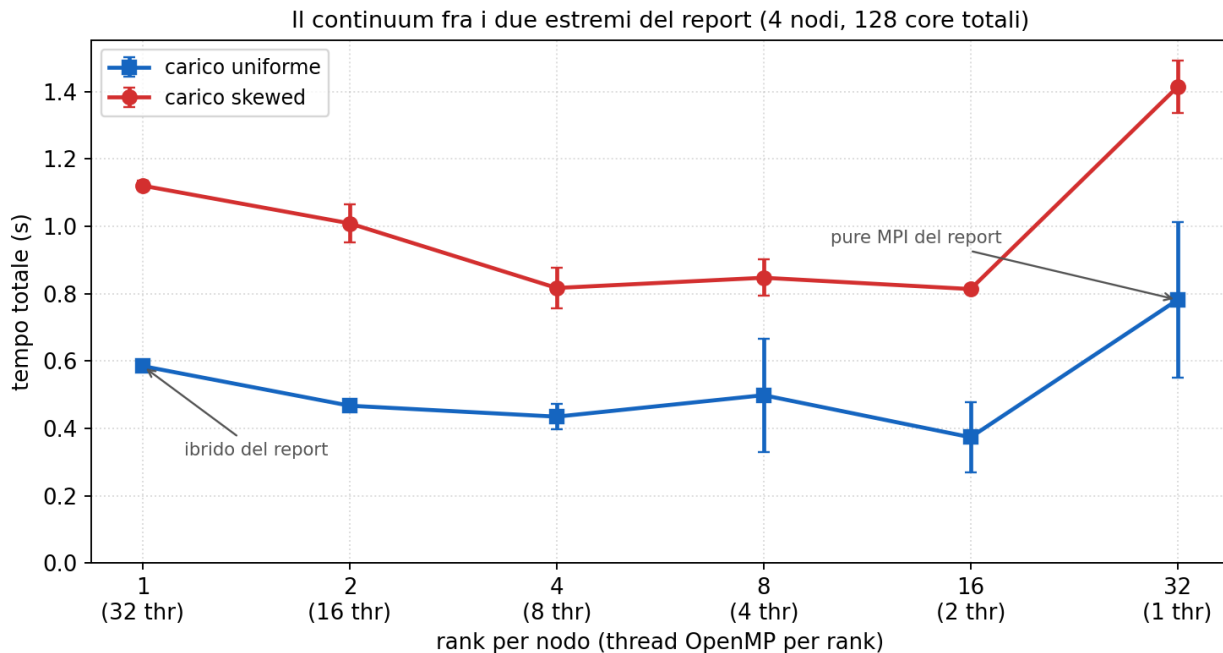
Verifica del modello sul weak scaling: domina il volume per scheda di rete, non il numero di startup



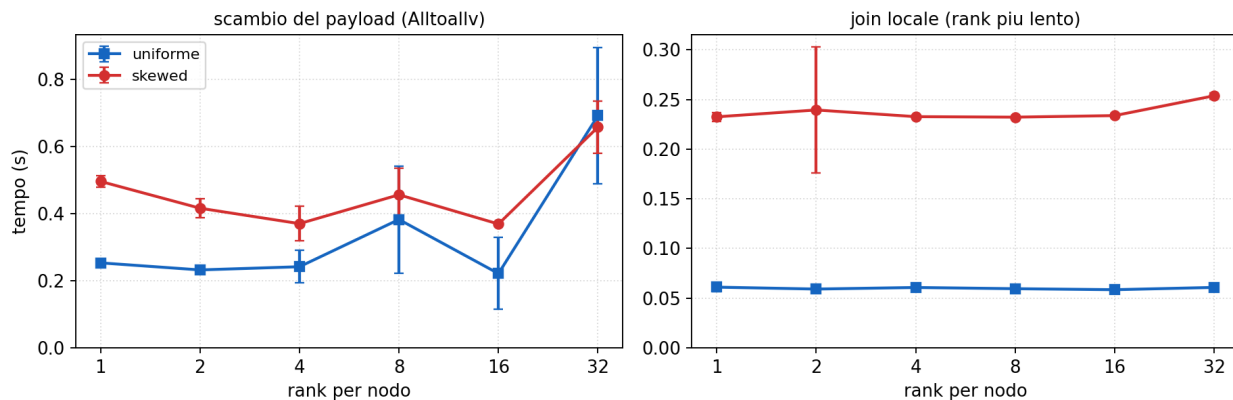
Ping-pong (common/pingpong_bench.cpp) fra 2 rank su nodi diversi e sullo stesso nodo, 8 B - 32 MB: alpha = 22.4 us, banda 1.15 GB/s inter-nodo (da 10 GbE); 0.6 us e 4.3 GB/s intra-nodo. Verifica sul weak scaling dell'Alltoallv (dati esp. 1):

- Modello del report, (R-1) alpha + beta V_rank: piatto a ~48 ms, due ordini di grandezza sotto il misurato a 256 rank (3.1 s). Gli startup non sono il costo.
- Modello a volume per NIC, beta x (rpn x V_rank x quota off-node): segue la curva entro un fattore 2-3 (il resto è contesa/incast). A 8 rank (1 per nodo) predice 42 ms contro 45 misurati: esatto.
- Take-away: l'ibrido vince non perché fa meno startup ma perché inietta meno traffico per scheda di rete a parità di payload globale... in realtà il payload globale è lo stesso: la differenza vera è il numero di partecipanti alla collettiva e la quota di scambi che restano dentro il nodo (31/32 dei destinatari di un rank sono remoti a 32 rpn, 7/8 dei nodi a 1 rpn ma con un solo rank a iniettare).

Esp. 3: il continuum rank per nodo (03_rankspernode/)



Le due componenti lungo il continuum

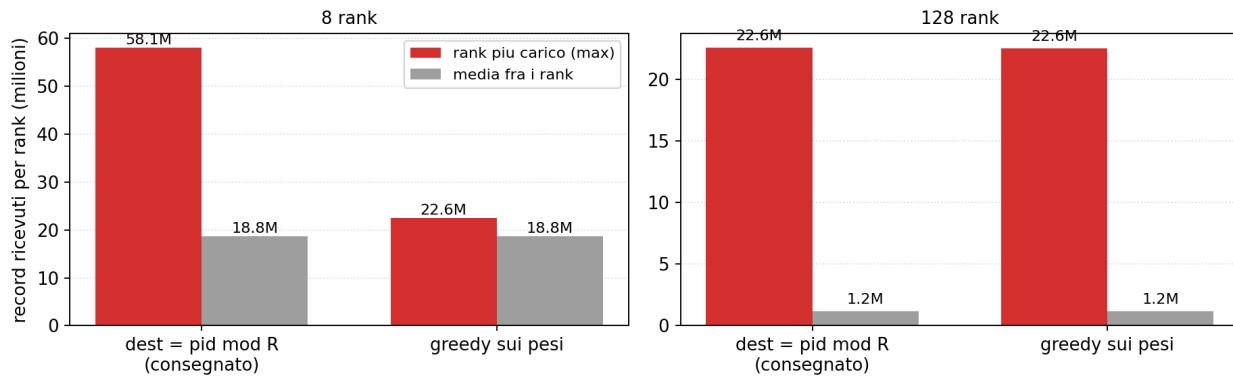


Driver `common/threadlevel_bench.cpp` (riusa `include/mpi_pipeline.hpp` consegnato, livello FUNNELED): a 4 nodi, rank per nodo in {1,2,4,8,16,32} con thread complementari ($rpn \times T = 32$). Risultato: il minimo è in mezzo, non agli estremi.

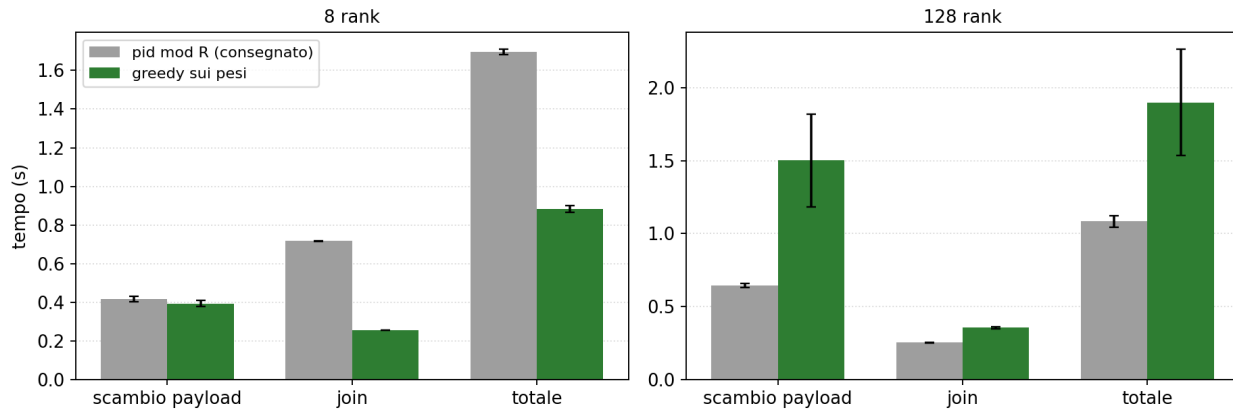
- Uniforme: 16 rpn x 2 thread = 0.37 s contro 0.59 (1 rpn, l'ibrido del report) e 0.78 (32 rpn, il pure MPI). Skewed: 4-16 rpn a 0.81-0.85 s contro 1.12 e 1.41.
- Meccanismo: lo scambio resta economico fino a 64 rank totali (la soglia critica è 128, esp. 1), e già 2-8 thread per rank bastano a parallelizzare le fasi locali. I due estremi pagano per intero uno dei due costi.
- Il join (max fra i rank) non dipende dalla configurazione: su skewed è 0.23-0.25 s ovunque, cioè l'imbalance non si cura spostando il confine rank/thread.
- 2 rank per nodo (uno per socket) era il candidato NUMA-naturale: è buono (0.47/1.01 s) ma non il migliore; il fattore dominante qui è il rank count della collettiva, non la località NUMA dentro il nodo.

Esp. 4: imbalance e remapping (04_remap_imbalance/)

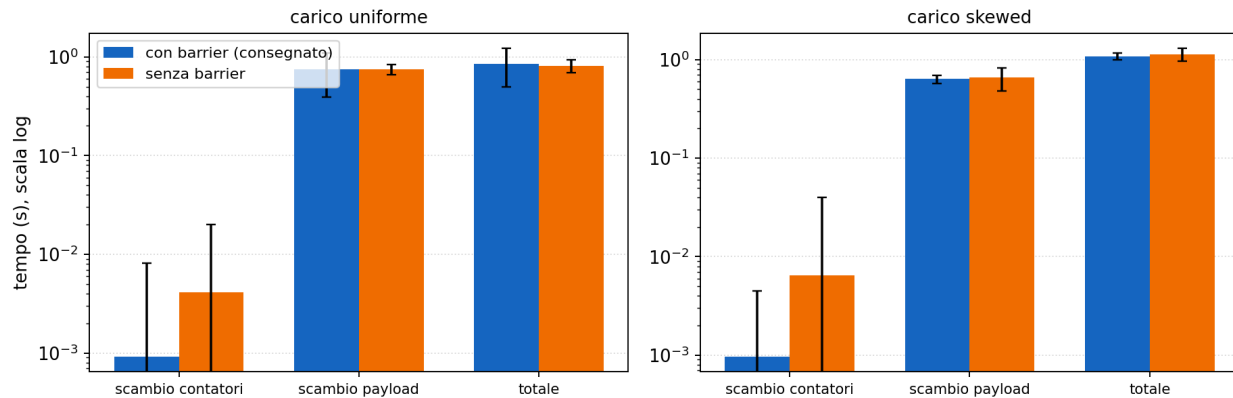
Volume ricevuto per rank sotto skew: a 128 rank il pavimento e la partizione indivisibile (22.6M record), nessun mapping puo scendere sotto



Effetto del remapping sui tempi (skew): grande a 8 rank, nullo o negativo a 128



Metodologia di timing: MPI_Barrier davanti alle collettive (128 rank, mod)



common/mpi_remap.cpp: pipeline pura MPI autonoma con mappa partizione-rank parametrica (-remap mod|greedy) e barrier disattivabili (-barrier 0). Il greedy: istogramma globale dei pesi per partizione (un Allreduce su P contatori, misurato 2-3 ms), ordinamento per peso decrescente, ogni partizione al rank più scarico (LPT sui rank); il piano è deterministico e identico su tutti i rank.

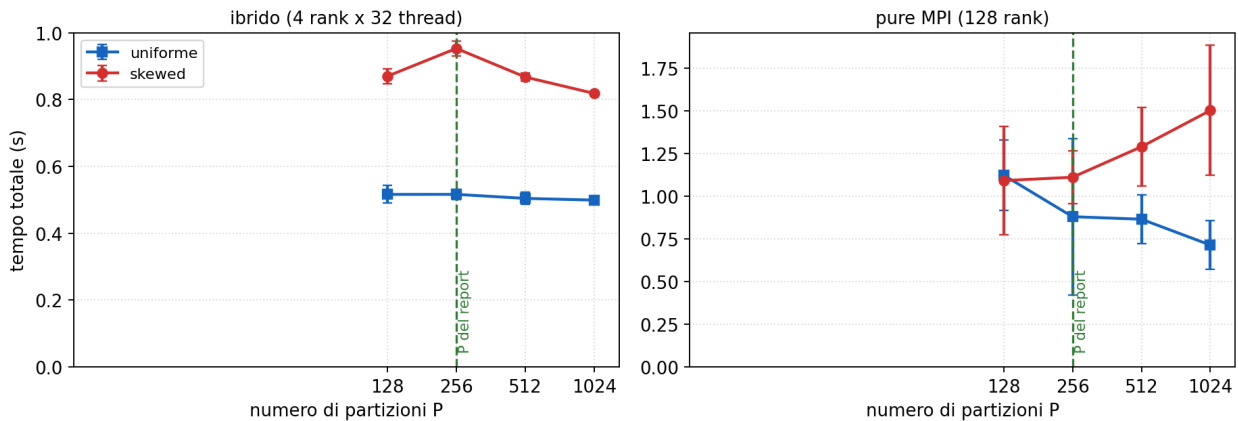
- Imbalance misurato (la cifra che mancava al report): 128 rank skewed, recv_max 22.6M contro media 1.2M

(19x). Il pavimento è la partizione hot di S (22.5M), indivisibile.

- A 8 rank il mod collide (58.1M sul rank peggiore) e il greedy ripara: totale da 1.70 a 0.88 s. A 128 rank il greedy non può nulla (pavimento) e peggiora la collettiva (send count irregolari: comm_payload da 0.64 a 1.50 s).
- Conclusione da difendere all'orale: il remapping è la risposta giusta alle COLLISIONI (pochi rank), non allo skew intrinseco (la partizione indivisibile), per il quale serve la replicazione del build side che il report discute e motiva di non aver fatto.
- Barrier on/off: totale entro il 5%, comm_sizes assorbe 3-6 ms senza barrier. La metodologia del report regge.

Esp. 5: sensibilità a P (05_p_sweep/)

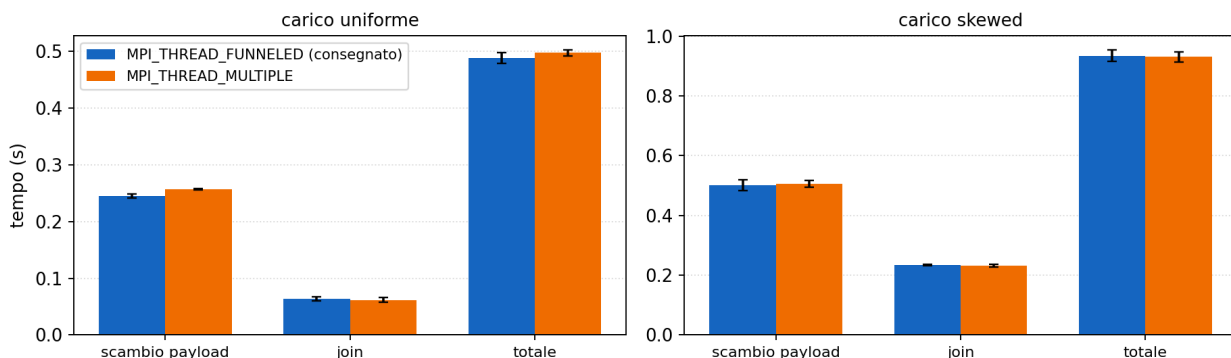
Sensibilità al numero di partizioni (4 nodi)



- Ibrido (4 rank x 32 thread): totale piatto su uniforme (0.50-0.52 s); il join scende da 74 a 40 ms alzando P a 1024 (tabelle in cache, stesso fenomeno dei moduli 2-3).
- Pure MPI (128 rank): su uniforme P=1024 vale 0.71 s contro 1.12 di P=128 (scambio più regolare e join in cache); su skewed oltre P=256 peggiora (1.50 s a P=1024: la collettiva paga i send count sbilanciati dello skew su più partizioni).
- P=256 del report: compromesso unico che rispetta P multiplo di 256 rank e non penalizza nessuna configurazione.

Esp. 6: FUNNELED vs MULTIPLE (06_threadlevel/)

Livello di thread support MPI: differenza dentro il rumore
(ibrido, 4 nodi, 4 rank x 32 thread, collettive fuori dalle regioni OpenMP)



Stessa pipeline, livello richiesto a runtime (`-threadlevel`), livello fornito verificato nell'output (`funneled/multiple`). Differenze dentro il rumore su entrambi i carichi. Vedi sez. 1 per la formulazione onesta.

4. Deep dive: MPI nel codice

Layout dest-major. `dest_major_pid(pid) = dest * P_per_rank + lp` con `dest = pid % R`, `lp = pid / R`: una permutazione di $[0, P)$ che rende il buffer di invio contiguo per destinatario, requisito di `MPI_Alltoallv` (un blocco per rank, descritto da counts e displacements). Il vincolo $P \bmod R = 0$ mantiene lo stesso numero di partizioni per rank.

Perché due histogram e due scatter (pre e post)? Il primo passaggio ordina per DESTINATARIO (per lo scambio); dopo lo scambio i record di un rank arrivano mescolati fra le sue P/R partizioni, e il kernel del join vuole il layout per-partizione: serve un secondo histogram+scatter sul buffer ricevuto (chiave $\rightarrow$ pid $\rightarrow$ lp = pid / R). È il prezzo del riuso del kernel M3 senza modifiche.

Perché Alltoallv e non Isend/Irecv con overlap? Il guadagno massimo è il lavoro post-scambio sovrapponibile: ~100 ms su 1.3 s (pure MPI, ~7%) o ~120 ms su 0.61 s (ibrido, ~19%) a 4 nodi. Il costo: spezzare la collettiva in scambi per-partizione, perdendo lo scheduling interno dell'implementazione tuned. Guadagno incerto e limitato contro una ristrutturazione certa: scelta documentata nel report come lavoro futuro. (Nota dall'esp. 1: con pairwise forzato la collettiva scende così tanto che il margine per l'overlap si riduce ulteriormente.)

Allreduce finale. Tre uint64 per rank, riduzione ad albero in $O(\log R)$ round: sotto il millisecondo sempre (misurato 1-6 ms includendo il barrier di allineamento).

Generazione per fette. Uniforme: skip-ahead $O(1)$ (stato = seed + GOLDEN*offset), nessuno scatter iniziale, nessun nodo tiene mai l'intera relazione. Skewed: rigenerazione completa e taglio della fetta (rejection sampling delle hot key rende l'offset ignoto), fuori dalla regione misurata; costa memoria e tempo di setup, non tempo misurato.

5. Onestà: le sfumature dove conviene essere precisi

1. **Il modello di Hockney del report ha il termine sbagliato in evidenza.** Con i coefficienti misurati gli startup valgono millisecondi, non secondi: a dominare è il volume per NIC (che cresce con i rank per nodo) più la contesa. La conclusione non cambia, il meccanismo sì.
 2. **Il dip a 128 rank aveva una cura da un parametro:** pairwise forzato (3x più veloce e stabile a 8 nodi). Il report ha diagnosticato correttamente la selezione dell'algoritmo ma ha provato solo basic linear; dirlo come "diagnosi giusta, esplorazione incompleta".
 3. **Gli estremi del report non sono l'ottimo:** a 4 nodi la configurazione migliore è 16 rank x 2 thread (0.37 contro 0.59 s dell'ibrido su uniforme). Il report confronta i due modelli canonici; il continuum mostra che il vero parametro è il rank count totale della collettiva.
 4. **Il remapping "possibile lavoro futuro" ora ha numeri:** utile solo contro le collisioni a basso rank count; a 128 rank è addirittura dannoso. La replicazione del build side resta l'unica via oltre il pavimento, come il report sostiene.
 5. **MULTIPLE non costa nulla di misurabile qui.** L'argomento "locking overhead" è potenziale, non misurato: formularlo come igiene del contratto di concorrenza.
 6. **Le partizioni hot sono fino a 8 (4 di R + 4 di S), non 4.** Il report parla di "four hot keys" per relazione; ai fini del floor conta la più pesante (hot di S, 22.5M record = 15% dell'input globale).
-

6. Cheat-sheet orale M4 (numeri da ricordare)

- Nodi: E5-2640 v2, 32 CPU logiche, 10 GbE (misurato: alpha 22 us, banda 1.15 GB/s inter-nodo; 4.3 GB/s intra). Input 1.2 GB: NR=50M, NS=100M, P=256, max_key=25M.
- Baseline seq: 4.48 s uniforme, 2.30 s skewed (località scatter 3.4x, tabelle hot in cache).

- Strong uniforme: ibrido 2.8x -> 12.0x (1 -> 8 nodi); pure MPI 6.9x, 7.6x, 3.0x, 7.1x. Dip a 128 rank: comm_payload 0.09/0.29/1.35/0.53 s. Sintetico: 64 rank 0.12 s, 128 rank 0.43 s. Pairwise forzato: 0.13 s a 8 nodi (3x sul default, stabile).
- Weak: ibrido 0.58, pure 0.21 a 8 nodi; payload ibrido 15 -> 52 ms, pure 0.12 -> 3.17 s. Startup (R-1) alpha = 5.6 ms a 256 rank: non spiegano; volume per NIC sì.
- Continuum (4 nodi): migliore 16 rpn x 2 thr = 0.37 s (uniforme); estremi 0.59 e 0.78 s. Skew: 4-16 rpn ~0.82 s contro 1.12 / 1.41.
- Skew floor: recv_max 22.6M vs media 1.2M (19x) a 128 rank; a 8 rank mod collide (58M), greedy ripara (totale 1.70 -> 0.88 s); a 128 rank greedy inutile sul join e -75% sul totale.
- Ibrido: prima iterazione 6.25 s a 1 nodo (Amdahl), tutte le fasi parallele 1.61 s; Alltoallv 256 -> 8 rank: 0.53 -> 0.18 s.
- Cross-module: uniforme M3 1 nodo 10.1x vs M4 8 nodi 12.0x; skew M3 6.2x vs M4 2.5x. Distribuzione = capacità, non velocità, su questo input.
- Validazione: 46 configurazioni PASS; naive $O(N^2)$ indipendente sui piccoli; vincolo $P \geq R$; skip-ahead split-mix64 (stato += GOLDEN per estrazione).